

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. **(5 marks)**

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

Code of Conduct:

I M.Vineela(192472071) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Vineela

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Predict the Reinforcement Learning (RL) Framework by Describing the Agent–Environment Interaction

Explanation

Reinforcement Learning (RL) is a machine learning technique where an **agent** learns by interacting with an **environment**. The agent performs actions and receives rewards based on those actions. Its objective is to maximize the total reward over time.

The interaction follows these steps:

1. The environment provides the current **state**.
2. The agent observes the state.
3. The agent chooses an **action**.
4. The environment moves to a new state.
5. The environment returns a **reward**.
6. The agent learns from the reward and repeats the process.

Key Components

- **Agent:** Learns and makes decisions.
- **Environment:** The world in which the agent operates.
- **State (S):** Current situation of the environment.
- **Action (A):** Decision taken by the agent.
- **Reward (R):** Feedback received after an action.

Cumulative Reward

Instead of maximizing only the immediate reward, the agent tries to maximize the **cumulative reward**, which is the sum of current and future rewards. This helps the agent make better long-term decisions.

Python Program:

```
state = 0
goal = 5
total_reward = 0
print("Reinforcement Learning Simulation")
while state != goal:
    action = input("Move (R/L): ").upper()
    if action == "R":
        state += 1
    elif action == "L":
        state -= 1
    if state == goal:
        reward = 10
    else:
        reward = -1
    total_reward += reward
    print("State:", state)
    print("Reward:", reward)
    print("Total Reward:", total_reward)
    print()
print("Learning Completed")
print("Final Reward:", total_reward)
```

Output:

```
state = 0
goal = 5
total_reward = 0

print("Reinforcement Learning Simulation")

while state != goal:
    action = input("Move (R/L): ").upper()

    if action == "R":
        state += 1
    elif action == "L":
        state -= 1

    if state == goal:
        reward = 10
        print("Goal Reached")
    else:
        reward = -1

    total_reward += reward

    print("State:", state)
    print("Reward:", reward)
    print("Total Reward:", total_reward)
    print()

print("Learning Completed")
print("Final Reward:", total_reward)
```

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
=====
Reinforcement Learning Simulation
Move (R/L): R
State: 1
Reward: -1
Total Reward: -1

Move (R/L): L
State: 0
Reward: -1
Total Reward: -2

Move (R/L): R
State: 1
Reward: -1
Total Reward: -3

Move (R/L): R
State: 2
Reward: -1
Total Reward: -4

Move (R/L): R
State: 3
Reward: -1
Total Reward: -5

Move (R/L): L
State: 2
Reward: -1
Total Reward: -6

Move (R/L):
```

2. Estimate How Decision-Making Problems Can Be Modeled as a Markov Decision Process (MDP) (5 Marks)

Explanation

A Markov Decision Process (MDP) is a mathematical framework used in Reinforcement Learning to model sequential decision-making problems.

An MDP consists of the following components:

- **States (S):** All possible situations.
- **Actions (A):** Choices available to the agent.
- **Transition Probability (P):** Probability of moving from one state to another.
- **Reward Function (R):** Reward received after an action.
- **Discount Factor (γ):** Determines the importance of future rewards.

Example

A robot moving in a grid:

- States $\rightarrow$ Grid positions.
- Actions $\rightarrow$ Up, Down, Left, Right.
- Reward $\rightarrow$ +10 for reaching goal, -1 otherwise.
- Transition Probability $\rightarrow$ Probability of successful movement.
- Discount Factor $\rightarrow$ Balances immediate and future rewards.

Conclusion

MDPs help RL agents make optimal decisions by considering current states, future transitions, and long-term rewards.

Python Program :

```
states = ["A", "B", "Goal"]

gamma = float(input("Enter Discount Factor (0 to 1): "))

current = 0

total_reward = 0

print("\n===== Markov Decision Process Simulation =====")

while current < len(states) - 1:

    print("\nCurrent State:", states[current])

    print("Available Actions: Right, Left")

    action = input("Enter Action: ")

    if action.lower() == "right":

        reward = 5

        transition_probability = 0.9

        current += 1

        print("Transition Successful")

    else:

        reward = -2

        transition_probability = 0.3

        print("Stayed in Same State")

    discounted_reward = reward * gamma
```

```

total_reward += discounted_reward

print("Transition Probability:", transition_probability)

print("Immediate Reward:", reward)

print("Discounted Reward:", round(discounted_reward, 2))

print("Total Reward:", round(total_reward, 2))

print("\n===== Goal State Reached =====")

print("Final State:", states[current])

print("Total Discounted Reward:", round(total_reward, 2))

```

Output:

The image shows a screenshot of a code editor (IDLE Shell 3.13.7*) displaying the execution of a Python script named EXP-1.py. The script implements a Markov Decision Process simulation with states 'A', 'B', and 'Goal'. It starts at state 'A' and allows actions 'Right' and 'Left'. The simulation proceeds through several steps, calculating immediate, discounted, and total rewards. The output shows the state transitions and rewards for each step, ending with the goal state reached.

```

states = ["A", "B", "Goal"]
gamma = float(input("Enter Discount Factor (0 to 1): "))

current = 0
total_reward = 0

print("\n===== Markov Decision Process Simulation =====")

while current < len(states) - 1:

    print("\nCurrent State:", states[current])

    print("Available Actions: Right, Left")
    action = input("Enter Action: ")

    if action.lower() == "right":
        reward = 5
        transition_probability = 0.9
        current += 1
        print("Transition Successful")
    else:
        reward = -2
        transition_probability = 0.3
        print("Stayed in Same State")

    discounted_reward = reward * gamma
    total_reward += discounted_reward

    print("Transition Probability:", transition_probability)
    print("Immediate Reward:", reward)
    print("Discounted Reward:", round(discounted_reward, 2))
    print("Total Reward:", round(total_reward, 2))

print("\n===== Goal State Reached =====")
print("Final State:", states[current])
print("Total Discounted Reward:", round(total_reward, 2))

```

```

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/EXP-1.py =====
Enter Discount Factor (0 to 1): 0.9

===== Markov Decision Process Simulation =====

Current State: A
Available Actions: Right, Left
Enter Action: R
Stayed in Same State
Transition Probability: 0.3
Immediate Reward: -2
Discounted Reward: -1.8
Total Reward: -1.8

Current State: A
Available Actions: Right, Left
Enter Action: L
Stayed in Same State
Transition Probability: 0.3
Immediate Reward: -2
Discounted Reward: -1.8
Total Reward: -3.6

Current State: A
Available Actions: Right, Left
Enter Action: R
Stayed in Same State
Transition Probability: 0.3
Immediate Reward: -2
Discounted Reward: -1.8
Total Reward: -5.4

Current State: A
Available Actions: Right, Left
Enter Action: |

```

3. Discuss the Role of Policy and Value Functions in RL and Examine State-Value and Action-Value Functions Using the Bellman Equation

Explanation

In Reinforcement Learning, a **policy** defines how an agent chooses actions in different states. It guides the agent toward maximizing long-term rewards.

A **value function** estimates how good a state or action is based on expected future rewards.

Types of Value Functions

State-Value Function ($V(s)$)

- Measures the expected cumulative reward from a state.
- Helps determine whether a state is beneficial.

Action-Value Function ($Q(s,a)$)

- Measures the expected cumulative reward after taking an action in a state.
- Helps the agent select the best action.

Bellman Equation

The Bellman Equation updates value estimates by combining the immediate reward with the discounted future reward.

State Value

$$V(s) = R + \gamma V(s')$$

Action Value

$$Q(s,a) = R + \gamma \max Q(s',a')$$

where:

- **R** = Immediate reward
- **γ** = Discount factor
- **V(s')** = Next state value
- **Q(s',a')** = Best future action value

Importance

- Evaluates long-term benefits.
- Helps improve policies.
- Supports optimal decision-making.

Python Program:

```
gamma = 0.9
rewards = [2, 4, 8]
state_values = [0, 0, 0]
print("Bellman Value Update")
for i in range(len(rewards)-1, -1, -1):
    if i == len(rewards)-1:
        state_values[i] = rewards[i]
    else:
        state_values[i] = rewards[i] + gamma * state_values[i+1]
for i in range(len(state_values)):
    print("State", i)
    print("Value =", round(state_values[i],2))
```

```
best_action = rewards.index(max(rewards))
print("\nBest Action:", best_action)
print("Action Value:", rewards[best_action])
```

Output:

```
gamma = 0.9
rewards = [2, 4, 8]
state_values = [0, 0, 0]

print("Bellman Value Update")
for i in range(len(rewards)-1, -1, -1):
    if i == len(rewards)-1:
        state_values[i] = rewards[i]
    else:
        state_values[i] = rewards[i] + gamma * state_values[i+1]

for i in range(len(state_values)):
    print("State", i)
    print("Value =", round(state_values[i],2))

best_action = rewards.index(max(rewards))

print("\nBest Action:", best_action)
print("Action Value:", rewards[best_action])
```

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/EXP-1.py ===
=====
Bellman Value Update
State 0
Value = 12.08
State 1
Value = 11.2
State 2
Value = 8

Best Action: 2
Action Value: 8

>>>
```


4. Justify the Exploration–Exploitation Trade-off in RL and Compare ϵ -Greedy and Softmax Approaches

Explanation:

In Reinforcement Learning (RL), an agent must decide between exploration and exploitation while interacting with the environment.

- **Exploration** means trying different actions to discover better rewards.
- **Exploitation** means selecting the action that currently gives the highest reward.

If the agent always exploits, it may miss better actions. If it always explores, it wastes time trying poor actions. Therefore, RL requires a balance between both.

ϵ -Greedy Strategy

- Chooses the best-known action most of the time.
- With probability ϵ (**epsilon**), it selects a random action.
- Example: If $\epsilon = 0.2$, then 20% of the time the agent explores and 80% it exploits.
- It is simple, efficient, and widely used.

Softmax Strategy

- Selects actions based on probabilities calculated from their estimated rewards.
- Better actions have higher probabilities, but weaker actions can still be selected.
- It explores more intelligently than ϵ -greedy.
- Controlled using a **temperature (T)** parameter:
 - High T $\rightarrow$ More exploration.

- Low T → More exploitation.

Comparison

ϵ-Greedy	Softmax
Random exploration	Probability-based exploration
Very simple	More intelligent
Fast to implement	Slightly more computation
Equal chance for all random actions	Better actions selected more often

Python program:

```

import random
import math
rewards = [4, 7, 10]
epsilon = float(input("Enter epsilon value (0 to 1): "))
temperature = float(input("Enter temperature value: "))
print("\n----- Epsilon-Greedy -----")
if random.random() < epsilon:
    action = random.randint(0, len(rewards)-1)
    print("Exploration")
else:
    action = rewards.index(max(rewards))
    print("Exploitation")
print("Selected Action:", action)
print("Reward:", rewards[action])
print("\n----- Softmax -----")
exp_values = []
for r in rewards:
    exp_values.append(math.exp(r / temperature))

```

```

total = sum(exp_values)
probabilities = []
for value in exp_values:
probabilities.append(value / total)
for i in range(len(probabilities)):
print("Action", i, "Probability =", round(probabilities[i], 3))
best_action = probabilities.index(max(probabilities))
print("Selected Action:", best_action)
print("Reward:", rewards[best_action])

```

Output:

```

EXP-1.py
import random
import math
rewards = [4, 7, 10]
epsilon = float(input("Enter epsilon value (0 to 1): "))
temperature = float(input("Enter temperature value: "))
print("\n----- Epsilon-Greedy -----")
if random.random() < epsilon:
    action = random.randint(0, len(rewards)-1)
    print("Exploration")
else:
    action = rewards.index(max(rewards))
    print("Exploitation")
print("Selected Action:", action)
print("Reward:", rewards[action])
print("\n----- Softmax -----")
exp_values = []
for r in rewards:
    exp_values.append(math.exp(r / temperature))

total = sum(exp_values)

probabilities = []
for value in exp_values:
    probabilities.append(value / total)

for i in range(len(probabilities)):
    print("Action", i, "Probability =", round(probabilities[i], 3))

best_action = probabilities.index(max(probabilities))
print("Selected Action:", best_action)
print("Reward:", rewards[best_action])

```

```

IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/EXP-1.py =====
Enter epsilon value (0 to 1): 0.2
Enter temperature value: 2

----- Epsilon-Greedy -----
Exploitation
Selected Action: 2
Reward: 10

----- Softmax -----
Action 0 Probability = 0.039
Action 1 Probability = 0.175
Action 2 Probability = 0.786
Selected Action: 2
Reward: 10
>>>

```

5. Explain How Reward Shaping Accelerates Learning in RL and Illustrate Its Impact on Preventing Suboptimal Policies

Explanation

Reward Shaping is a technique in Reinforcement Learning where additional rewards or penalties are given to the agent to guide it toward the desired behavior.

Normally, an agent receives a reward only after reaching the final goal. This can make learning slow because the agent does not know whether intermediate actions are good or bad.

Reward shaping provides extra feedback during learning. For example:

- Give **+1** for moving closer to the goal.
- Give **-1** for moving away from the goal.
- Give a large reward such as **+10** for reaching the goal.

This helps the agent learn faster by encouraging useful actions and discouraging poor actions.

Advantages

- Speeds up learning.
- Improves convergence.
- Reduces unnecessary exploration.
- Helps avoid suboptimal policies.
- Improves cumulative reward.

Python Program :

```
goal = int(input("Enter Goal Position: "))
position = int(input("Enter Starting Position: "))
total_reward = 0
steps = 0
print("\n===== Reward Shaping Simulation =====")
while position != goal:
    print("\nCurrent Position:", position)
    action = input("Move (R/L): ").upper()
    if action == "R":
        position += 1
    elif action == "L":
        position -= 1
    else:
        print("Invalid Action")
        continue
    if position == goal:
        reward = 10
        print("Goal Reached!")
    elif position < goal:
        reward = 2
        print("Moving Toward Goal")
    else:
        reward = -2
        print("Moving Away From Goal")
    total_reward += reward
    steps += 1
```

```

print("Reward:", reward)
print("Total Reward:", total_reward)
print("Steps:", steps)

print("\n===== Learning Completed =====")
print("Final Position:", position)
print("Total Reward:", total_reward)
print("Total Steps:", steps)

```

Output:

The image shows a screenshot of a Python script named 'EXP-1.py' and its execution output in the IDLE Shell. The script implements a simple reinforcement learning simulation where a goal is set, and a position is updated based on user input (R for right, L for left). The simulation tracks total reward and steps until the goal is reached or the learning is completed.

```

goal = int(input("Enter Goal Position: "))
position = int(input("Enter Starting Position: "))

total_reward = 0
steps = 0

print("\n===== Reward Shaping Simulation =====")

while position != goal:

    print("\nCurrent Position:", position)

    action = input("Move (R/L): ").upper()

    if action == "R":
        position += 1
    elif action == "L":
        position -= 1
    else:
        print("Invalid Action")
        continue

    if position == goal:
        reward = 10
        print("Goal Reached!")
    elif position < goal:
        reward = 2
        print("Moving Toward Goal")
    else:
        reward = -2
        print("Moving Away From Goal")

    total_reward += reward
    steps += 1

    print("Reward:", reward)
    print("Total Reward:", total_reward)
    print("Steps:", steps)

print("\n===== Learning Completed =====")
print("Final Position:", position)

```

The output in the IDLE Shell shows the execution of the script. It prompts the user to enter the goal position (5) and the starting position (2). The simulation then proceeds with several moves, showing the current position, the action taken, the reward received, the total reward, and the number of steps taken. The simulation ends when the goal is reached, displaying the final position, total reward, and total steps.

```

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/EXP-1.py =====
Enter Goal Position: 5
Enter Starting Position: 2

===== Reward Shaping Simulation =====

Current Position: 2
Move (R/L): R
Moving Toward Goal
Reward: 2
Total Reward: 2
Steps: 1

Current Position: 3
Move (R/L): L
Moving Toward Goal
Reward: 2
Total Reward: 4
Steps: 2

Current Position: 2
Move (R/L): R
Moving Toward Goal
Reward: 2
Total Reward: 6
Steps: 3

Current Position: 3
Move (R/L): L
Moving Toward Goal
Reward: 2
Total Reward: 8
Steps: 4

Current Position: 2
Move (R/L):

```

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

Explanation:

Reinforcement Learning (RL) is widely used in robotics for tasks such as navigation, object manipulation, and autonomous control. However, applying RL in real-world robots is difficult because physical environments are more complex than simulations.

Challenges in Real-World Robotics

1. Safety

- Robots interact with real objects and humans.
- Wrong actions may damage the robot or surroundings.
- Safe exploration is required to avoid accidents.
- Example: A robot arm should not hit nearby objects while learning.

2. Sample Efficiency

- RL usually requires thousands of training episodes.
- Collecting data from real robots is expensive and time-consuming.
- Every trial consumes battery power and hardware resources.
- Therefore, algorithms should learn from fewer experiences.

3. Environment Variability

- Real environments continuously change.
- Lighting, obstacles, weather, and sensor noise affect robot performance.

- The learned policy should adapt to new situations instead of memorizing one environment.

Effect on Agent Performance

- Unsafe exploration reduces reliability.
- Poor sample efficiency increases training time.
- Environment variability decreases accuracy if the robot cannot generalize.
- Robust RL algorithms improve adaptability and long-term performance.

Python Code :

```
robot_name = input("Enter Robot Name: ")
safe_actions = int(input("Enter Safe Actions: "))
unsafe_actions = int(input("Enter Unsafe Actions: "))
episodes = int(input("Enter Training Episodes: "))
environment = input("Environment (Stable/Variable): ")
print("\n----- Robot RL Analysis -----")
print("Robot:", robot_name)
if unsafe_actions == 0:
    print("Safety Status : Excellent")
elif unsafe_actions <= 2:
    print("Safety Status : Moderate")
else:
    print("Safety Status : Poor")
if episodes < 100:
    print("Sample Efficiency : High")
elif episodes < 500:
    print("Sample Efficiency : Medium")
else:
```



```

print("Sample Efficiency : Low")
if environment.lower() == "stable":
print("Environment : Stable")
print("Agent Performance : Better Learning")
else:
print("Environment : Variable")
print("Agent Performance : Needs Adaptation")
score = safe_actions - unsafe_actions
print("Performance Score:", score)
if score >= 8:
print("Overall Performance : Excellent")
elif score >= 5:
print("Overall Performance : Good")
else:
print("Overall Performance : Needs Improvement")

```

Output:

```

EXP-1.py
robot_name = input("Enter Robot Name: ")
safe_actions = int(input("Enter Safe Actions: "))
unsafe_actions = int(input("Enter Unsafe Actions: "))
episodes = int(input("Enter Training Episodes: "))
environment = input("Environment (Stable/Variable): ")

print("\n---- Robot RL Analysis ----")
print("Robot:", robot_name)

if unsafe_actions == 0:
    print("Safety Status : Excellent")
elif unsafe_actions <= 2:
    print("Safety Status : Moderate")
else:
    print("Safety Status : Poor")

if episodes < 100:
    print("Sample Efficiency : High")
elif episodes < 500:
    print("Sample Efficiency : Medium")
else:
    print("Sample Efficiency : Low")

if environment.lower() == "stable":
    print("Environment : Stable")
    print("Agent Performance : Better Learning")
else:
    print("Environment : Variable")
    print("Agent Performance : Needs Adaptation")

score = safe_actions - unsafe_actions
print("Performance Score:", score)

if score >= 8:
    print("Overall Performance : Excellent")
elif score >= 5:
    print("Overall Performance : Good")
else:
    print("Overall Performance : Needs Improvement")

```

```

IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/EXP-1.py =====
>>> Enter Robot Name: Robo 1
Enter Safe Actions: 10
Enter Unsafe Actions: 1
Enter Training Episodes: 8
Environment (Stable/Variable): variable

---- Robot RL Analysis ----
Robot: Robo 1
Safety Status : Moderate
Sample Efficiency : High
Environment : Variable
Agent Performance : Needs Adaptation
Performance Score: 9
Overall Performance : Excellent
>>>

```

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks.

Explanation

The **discount factor** (γ) determines how much importance an RL agent gives to future rewards.

- γ (**Gamma**) ranges between **0** and **1**.

Short-Term Decision Making

- When γ is close to **0**, the agent focuses only on immediate rewards.
- Future rewards have very little influence.
- Suitable for tasks where immediate success is important.

Long-Term Decision Making

- When γ is close to **1**, the agent values future rewards.
- It may sacrifice immediate rewards to obtain larger future rewards.
- Useful in navigation, robotics, and strategic games.

Role in Episodic Tasks

- Episodic tasks have a clear beginning and ending.
- The agent maximizes the total reward before the episode ends.
- A higher γ encourages planning until the terminal state.
- A lower γ makes the agent greedy for immediate rewards.

Python Code :

```
gamma = float(input("Enter Discount Factor (0 to 1): "))

num_states = int(input("Enter Number of States: "))

rewards = []

print("\nEnter Reward for Each State")

for i in range(num_states):

    reward = float(input("Reward for State " + str(i + 1) + ": "))

    rewards.append(reward)

    discounted_reward = 0

    print("\n===== Bellman Reward Calculation =====")

    for i in range(num_states - 1, -1, -1):

        discounted_reward = rewards[i] + gamma * discounted_reward

        print("State", i + 1)

        print("Immediate Reward :", rewards[i])

        print("Discounted Reward :", round(discounted_reward, 2))

    print()

    print("===== Final Result =====")

    print("Total Discounted Reward:", round(discounted_reward, 2))

    if gamma < 0.5:

        print("Decision: Agent prefers Short-Term Rewards")
```

elif gamma < 0.8:

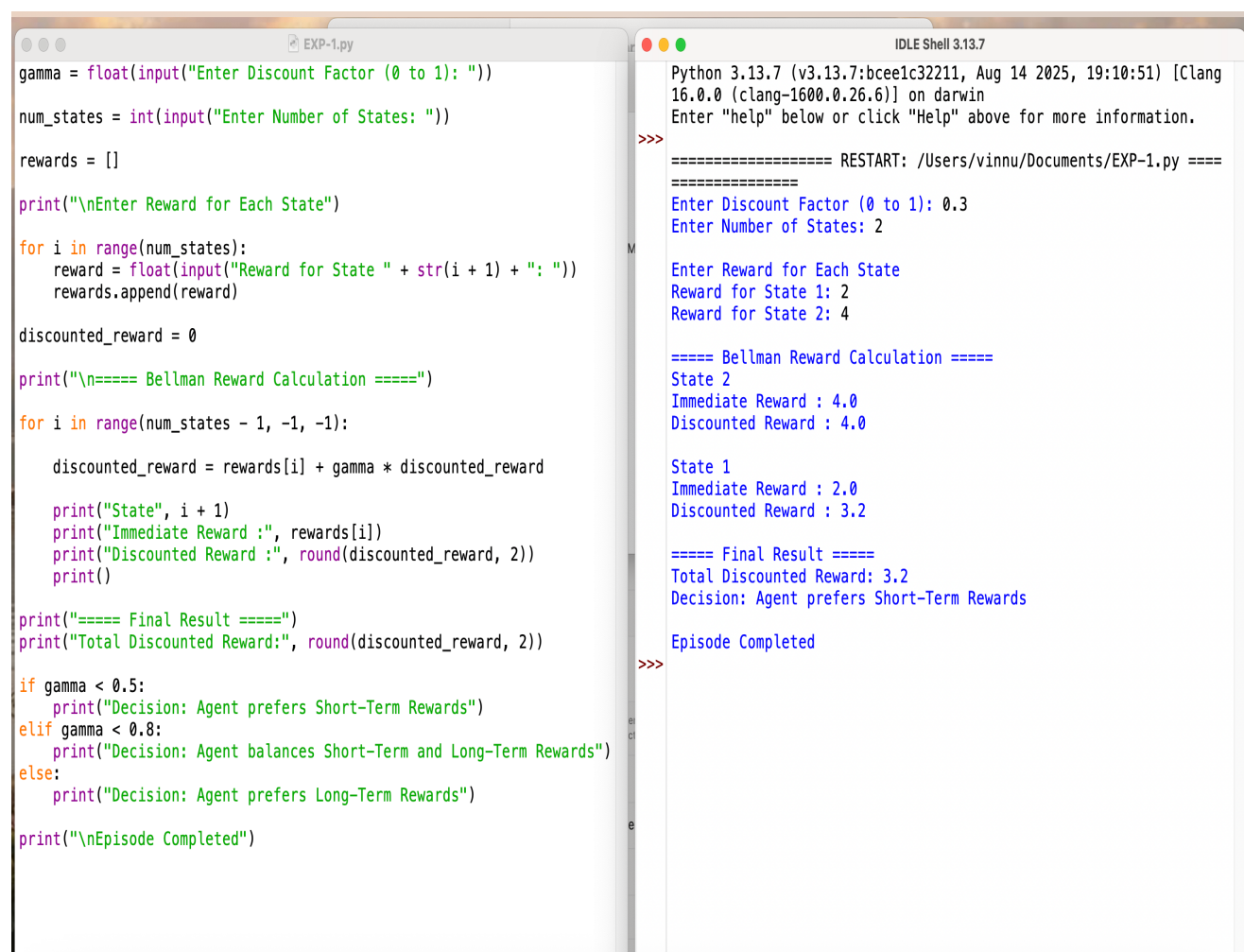
print("Decision: Agent balances Short-Term and Long-Term Rewards")

else:

print("Decision: Agent prefers Long-Term Rewards")

print("\nEpisode Completed")

Output:



The image shows a screenshot of a Python script named 'EXP-1.py' and its execution output in an IDLE Shell. The script calculates the Bellman reward for two states based on a discount factor gamma. The output shows the user input for gamma (0.3) and the number of states (2), followed by the calculation of rewards for each state and the final decision based on the discount factor.

```
gamma = float(input("Enter Discount Factor (0 to 1): "))
num_states = int(input("Enter Number of States: "))
rewards = []

print("\nEnter Reward for Each State")
for i in range(num_states):
    reward = float(input("Reward for State " + str(i + 1) + ": "))
    rewards.append(reward)

discounted_reward = 0
print("\n==== Bellman Reward Calculation =====")
for i in range(num_states - 1, -1, -1):
    discounted_reward = rewards[i] + gamma * discounted_reward

    print("State", i + 1)
    print("Immediate Reward :", rewards[i])
    print("Discounted Reward :", round(discounted_reward, 2))
    print()

print("==== Final Result =====")
print("Total Discounted Reward:", round(discounted_reward, 2))

if gamma < 0.5:
    print("Decision: Agent prefers Short-Term Rewards")
elif gamma < 0.8:
    print("Decision: Agent balances Short-Term and Long-Term Rewards")
else:
    print("Decision: Agent prefers Long-Term Rewards")

print("\nEpisode Completed")
```

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: /Users/vinnu/Documents/EXP-1.py =====>>>
Enter Discount Factor (0 to 1): 0.3
Enter Number of States: 2

Enter Reward for Each State
Reward for State 1: 2
Reward for State 2: 4

==== Bellman Reward Calculation =====
State 2
Immediate Reward : 4.0
Discounted Reward : 4.0

State 1
Immediate Reward : 2.0
Discounted Reward : 3.2

==== Final Result =====
Total Discounted Reward: 3.2
Decision: Agent prefers Short-Term Rewards

Episode Completed
>>>

8. Evaluate the effectiveness of Model-Free RL compared to Model-Based RL in dynamic environments, and differentiate their strengths and limitations.

Explanation

Reinforcement Learning algorithms are mainly divided into Model-Free RL and Model-Based RL.

Model-Free RL

- Learns directly through interaction with the environment.
- Does not know the environment model.
- Examples: Q-Learning, SARSA, Deep Q Network (DQN).

Strengths

- Easy to implement.
- Works well in unknown environments.
- No need for transition probabilities.

Limitations

- Requires many training samples.
- Slow convergence.
- High computational cost.

Model-Based RL

- Learns or uses a model of the environment.
- Predicts future states before taking actions.
- Examples: Value Iteration, Policy Iteration.

Strengths

- More sample efficient.
- Faster learning.
- Better planning capability.

Limitations

- Requires an accurate environment model.
- Difficult to build for complex systems.
- Performance drops if the model is inaccurate.

Dynamic Environments

- **Model-Free RL** adapts better to changing environments but learns slowly.
- **Model-Based RL** learns faster but struggles when the environment changes frequently because its model becomes outdated.

Python Code:

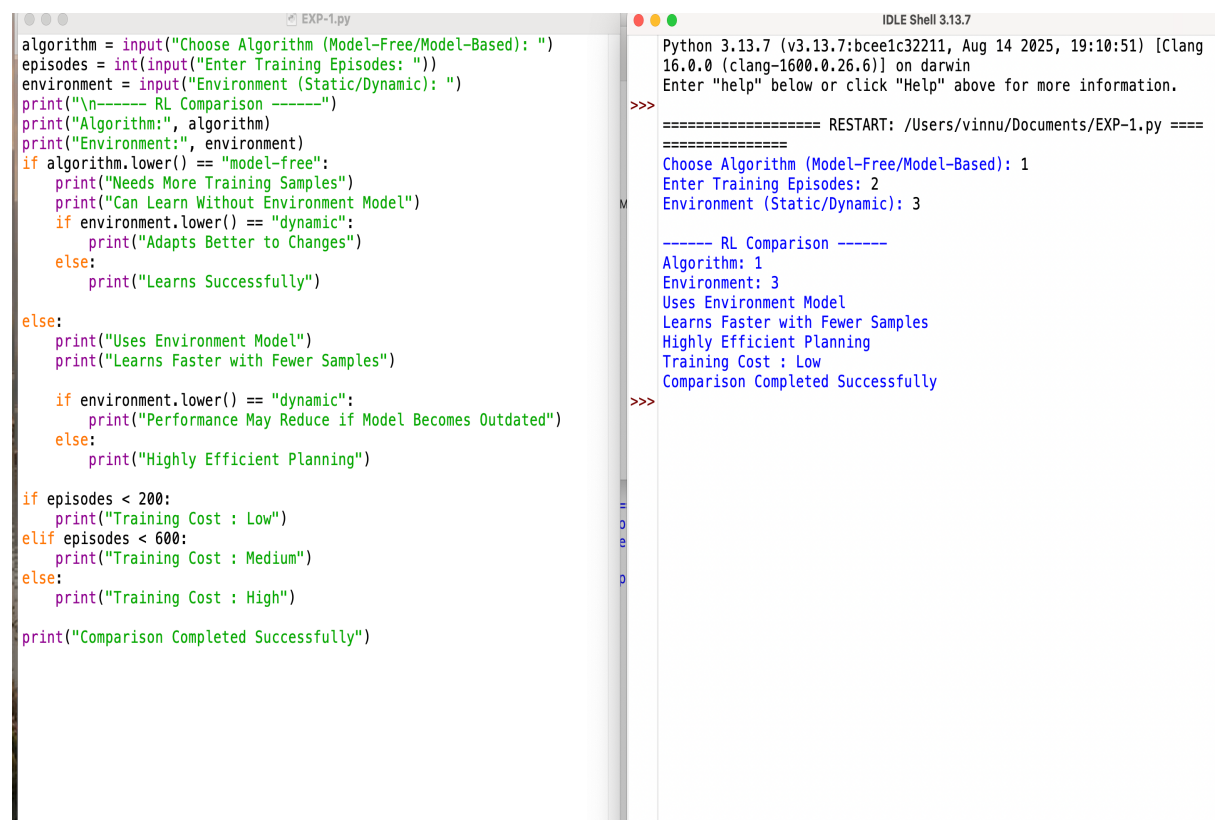
```
algorithm = input("Choose Algorithm (Model-Free/Model-Based): ")
episodes = int(input("Enter Training Episodes: "))
environment = input("Environment (Static/Dynamic): ")
print("\n----- RL Comparison -----")
print("Algorithm:", algorithm)
print("Environment:", environment)
if algorithm.lower() == "model-free":
    print("Needs More Training Samples")
    print("Can Learn Without Environment Model")
if environment.lower() == "dynamic":
    print("Adapts Better to Changes")
else:
```

```

print("Learns Successfully")
else:
print("Uses Environment Model")
print("Learns Faster with Fewer Samples")
if environment.lower() == "dynamic":
print("Performance May Reduce if Model Becomes Outdated")
else:
print("Highly Efficient Planning")
if episodes < 200:
print("Training Cost : Low")
elif episodes < 600:
print("Training Cost : Medium")
else:
print("Training Cost : High")
print("Comparison Completed Successfully")

```

Output:



The image shows a side-by-side comparison of a Python script and its execution output. On the left, a code editor window titled 'EXP-1.py' displays the following code:

```

algorithm = input("Choose Algorithm (Model-Free/Model-Based): ")
episodes = int(input("Enter Training Episodes: "))
environment = input("Environment (Static/Dynamic): ")
print("\n----- RL Comparison -----")
print("Algorithm:", algorithm)
print("Environment:", environment)
if algorithm.lower() == "model-free":
    print("Needs More Training Samples")
    print("Can Learn Without Environment Model")
    if environment.lower() == "dynamic":
        print("Adapts Better to Changes")
    else:
        print("Learns Successfully")
else:
    print("Uses Environment Model")
    print("Learns Faster with Fewer Samples")
    if environment.lower() == "dynamic":
        print("Performance May Reduce if Model Becomes Outdated")
    else:
        print("Highly Efficient Planning")
if episodes < 200:
    print("Training Cost : Low")
elif episodes < 600:
    print("Training Cost : Medium")
else:
    print("Training Cost : High")
print("Comparison Completed Successfully")

```

On the right, an 'IDLE Shell 3.13.7' window shows the output of the script. It starts with the Python version and date, followed by a prompt to enter help. The user has restarted the script, and the output shows the following sequence of prompts and responses:

```

===== RESTART: /Users/vinnu/Documents/EXP-1.py =====
Choose Algorithm (Model-Free/Model-Based): 1
Enter Training Episodes: 2
Environment (Static/Dynamic): 3

----- RL Comparison -----
Algorithm: 1
Environment: 3
Uses Environment Model
Learns Faster with Fewer Samples
Highly Efficient Planning
Training Cost : Low
Comparison Completed Successfully

```

9. Illustrate with an Example How Q-Learning Updates the Action-Value Function During Training and Demonstrate Its Convergence Behavior.

Explanation

Q-Learning is a model-free, off-policy Reinforcement Learning algorithm that helps an agent learn the optimal action for every state. It stores knowledge in a Q-table, where each value represents the expected future reward of taking a particular action in a given state.

Initially, all Q-values are set to 0. During training, the agent repeatedly interacts with the environment by selecting actions, receiving rewards, and moving to the next state. After every action, the Q-value is updated using the Q-learning update equation.

Q-learning Update Rule

where:

- $Q(s,a)$ → Current Q-value
- α (alpha) → Learning rate
- γ (gamma) → Discount factor
- R → Immediate reward
- s' → Next state

As the number of training episodes increases, the Q-values gradually stabilize. This is known as convergence. Once converged, the Q-table contains the best action-value estimates, allowing the agent to follow the optimal policy.

Advantages

- Learns optimal policy through experience.
- Does not require an environment model.
- Simple and widely used.

Python Program:

```
alpha = float(input("Enter Learning Rate (alpha): "))
gamma = float(input("Enter Discount Factor (gamma): "))
episodes = int(input("Enter Number of Episodes: "))
q_value = 0
print("\n===== Q-Learning Training =====")
for episode in range(1, episodes + 1)
    reward = float(input("\nEnter Reward for Episode " + str(episode) + ": "))
    next_q = float(input("Enter Maximum Next Q Value: "))
    old_q = q_value
    q_value = old_q + alpha * (reward + gamma * next_q - old_q)
    print("Old Q Value:", round(old_q,2))
    print("Updated Q Value:", round(q_value,2))
print("\n===== Training Completed =====")
print("Final Q Value:", round(q_value,2))
print("Q-value has gradually converged after multiple updates.")
Enter Maximum Next Q Value: 8
```

Output:

```
EXP-1.py
alpha = float(input("Enter Learning Rate (alpha): "))
gamma = float(input("Enter Discount Factor (gamma): "))
episodes = int(input("Enter Number of Episodes: "))

q_value = 0

print("\n==== Q-Learning Training =====")

for episode in range(1, episodes + 1):

    reward = float(input("\nEnter Reward for Episode " + str(episode)
    next_q = float(input("Enter Maximum Next Q Value: "))

    old_q = q_value

    q_value = old_q + alpha * (reward + gamma * next_q - old_q)

    print("Old Q Value:", round(old_q,2))
    print("Updated Q Value:", round(q_value,2))

print("\n==== Training Completed =====")
print("Final Q Value:", round(q_value,2))
print("Q-value has gradually converged after multiple updates.")

IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/EXP-1.py =====
Enter Learning Rate (alpha): 0.5
Enter Discount Factor (gamma): 0.9
Enter Number of Episodes: 2

==== Q-Learning Training =====

Enter Reward for Episode 1: 5
Enter Maximum Next Q Value: 4
Old Q Value: 0
Updated Q Value: 4.3

Enter Reward for Episode 2: 7
Enter Maximum Next Q Value: 8
Old Q Value: 4.3
Updated Q Value: 9.25

==== Training Completed =====
Final Q Value: 9.25
Q-value has gradually converged after multiple updates.

>>>
```

10. Differentiate Between On-Policy and Off-Policy Learning Methods in RL and Assess Their Advantages and Limitations

Explanation

Reinforcement Learning algorithms are mainly classified into **On-Policy** and **Off-Policy** learning methods based on how the agent learns from actions.

On-Policy Learning

In On-Policy learning, the agent learns using the **same policy** that it follows while interacting with the environment. The actions used for learning are the same actions used to make decisions.

Example: SARSA

Off-Policy Learning

In Off-Policy learning, the agent learns using a **different policy** from the one used to collect experience. It can learn from past experiences or another agent's actions.

Example: Q-Learning

Comparison

On-Policy	Off-Policy
Learns from current policy	Learns from another or optimal policy
More stable	More sample efficient
Safer exploration	Faster convergence
Less flexible	More flexible
Example: SARSA	Example: Q-Learning

Advantages of On-Policy

- Stable learning.
- Suitable for safety-critical tasks.
- Easier to understand.

Limitations of On-Policy

- Learns slowly.
- Requires more interactions.

Advantages of Off-Policy

- Faster learning.
- Reuses previous experiences.
- Higher sample efficiency.

Limitations of Off-Policy

- Can become unstable.
- Sensitive to parameter settings.

Python Program:

```
episodes = int(input("Enter Number of Episodes: "))
on_total = 0
off_total = 0
print("\n===== Reinforcement Learning Comparison =====")
for i in range(1, episodes + 1):
    on_reward = float(input("\nEnter On-Policy Reward for Episode " + str(i) + ": "))
    off_reward = float(input("Enter Off-Policy Reward for Episode " + str(i) + ": "))
```

```

on_total += on_reward
off_total += off_reward
on_average = on_total / episodes
off_average = off_total / episodes
print("\n===== Results =====")
print("Average On-Policy Reward :", round(on_average, 2))
print("Average Off-Policy Reward:", round(off_average, 2))
if on_average > off_average:
    print("On-Policy performed better.")
elif off_average > on_average:
    print("Off-Policy performed better.")
else:
    print("Both methods performed equally.")
print("\nExamples:")
print("On-Policy : SARSA")
print("Off-Policy: Q-Learning")

```

Output:

The image shows a side-by-side comparison of a Python script and its execution output. On the left, the script 'EXP-1.py' is displayed in a code editor. It defines variables for episodes, on-policy total reward, off-policy total reward, and their averages. It uses a loop to collect rewards for two episodes, then prints the results and examples. On the right, the 'IDLE Shell 3.13.7' window shows the output of the script. It prompts for the number of episodes (2), collects rewards for each episode (On-Policy: 4, 5; Off-Policy: 2, 6), and prints the calculated averages (On-Policy: 4.5, Off-Policy: 4.0), concluding that On-Policy performed better. It also lists the examples as SARSA for On-Policy and Q-Learning for Off-Policy.

```

EXP-1.py
episodes = int(input("Enter Number of Episodes: "))
on_total = 0
off_total = 0

print("\n===== Reinforcement Learning Comparison =====")
for i in range(1, episodes + 1):
    on_reward = float(input("\nEnter On-Policy Reward for Episode " + str(i) + ": "))
    off_reward = float(input("Enter Off-Policy Reward for Episode " + str(i) + ": "))
    on_total += on_reward
    off_total += off_reward

on_average = on_total / episodes
off_average = off_total / episodes

print("\n===== Results =====")
print("Average On-Policy Reward :", round(on_average, 2))
print("Average Off-Policy Reward:", round(off_average, 2))

if on_average > off_average:
    print("On-Policy performed better.")
elif off_average > on_average:
    print("Off-Policy performed better.")
else:
    print("Both methods performed equally.")

print("\nExamples:")
print("On-Policy : SARSA")
print("Off-Policy: Q-Learning")

```

```

IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/EXP-1.py =====
Enter Number of Episodes: 2

===== Reinforcement Learning Comparison =====

Enter On-Policy Reward for Episode 1: 4
Enter Off-Policy Reward for Episode 1: 2

Enter On-Policy Reward for Episode 2: 5
Enter Off-Policy Reward for Episode 2: 6

===== Results =====
Average On-Policy Reward : 4.5
Average Off-Policy Reward: 4.0
On-Policy performed better.

Examples:
On-Policy : SARSA
Off-Policy: Q-Learning
>>>

```